Автономная некоммерческая образовательная организация
"Физтех-лицей"имени П.Л. Капицы

УДК 004.056.5

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к проекту по предмету «информатика»
профиль: «Информационная Безопасность»
по теме:

«Разработка криптографического приложения, повышающего безопасность неопытного пользователя»

Выполнил:
Ученик 10 класса Завадский Алексей Дмитриевич

Долгопрудный 2026

РЕФЕРАТ

Отчет 31 с., 1 кн., 1 табл., 14 ист., 1 прил.

РИПТОГРАФИЯ, ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ,
ХЕШ-ФУНКЦИИ, RSA, ШИФРОВАНИЕ, OPENSSL, ЗАЩИТА ДАННЫХ.

Цель работы - разработка программного продукта, позволяющего выполнять базовые криптографические операции для повышения безопасности непродвинутого пользователя.

В процессе работы рассмотрены существующие решения и проведён их анализ, разработаны требования к создаваемому программному продукту, описана архитектура и реализованный функционал приложения. Особое внимание уделено удобству использования инструмента неподготовленными пользователями и предотвращению типичных ошибок, снижающих уровень безопасности.

В результате выполенения работы был разработан программный продукт, отвечающий всем поставленным требованиям и решающий все поставленные задачи. Он обеспечивает безопасность и конфиденциальность данных пользователя. Использование моего продукта поможет сократить количество инцидентов информационной безопасности.

Содержание

РЕФЕРАТ **1**

СПИСОК СОКРАЩЕНИЙ И ТЕРМИНОВ **4**

ВВЕДЕНИЕ **5**

1 ИССЛЕДОВАНИЕ ПРОБЛЕМЫ **6**

1.1 Предметная область 6

1.2 Проблема и практическая значимость 7

1.3 Сравнение с существующими аналогами 7

1.4 Аудитория проекта 9

2 РЕШЕНИЕ ПРОБЛЕМЫ **9**

2.1 Описание разрабатываемого приложения 9

2.1.1 Общее назначение системы 9

2.1.2 Основные функции 9

2.2 Функциональные возможности проекта 10

2.2.1 Шифрование и расшифрование файлов 10

2.2.2 Проверка сложности пароля 11

2.2.3 Вычисление и сравнение хешей 11

2.2.4 RSA 11

2.2.5 История операций 11

2.2.6 Настройки 12

2.2.7 Поддерживаемые и планируемые расширения 12

2.3 Модель угроз 12

2.3.1 Соответствие угроз техникам MITRE ATT&CK и меры противодействия 12

2.3.2 Вывод по модели угроз 15

3 АРХИТЕКТУРА И ПРИНЦИП РЕАЛИЗАЦИИ **15**

3.1 Техническое задание 15

3.1.1 Цели и задачи разработки 15

3.1.2 Технические требования 15

3.1.3 Описание выполняемых функций продукта 16
3.2 Логическая структура приложения 17
3.3 Организация пользовательского интерфейса 18
3.4 Взаимодействие с внешним процессом 18
3.5 Журналирование и приватность 18
3.6 Меры защиты, заложенные в проект 19
3.6.1 Защита от инъекции аргументов 19
3.6.2 Проверка целостности `openssl.exe` 19
3.6.3 Проверка наличия зависимости при запуске 19
3.6.4 Ограничение утечек через журналирование 20
3.6.5 Предупреждения о слабых параметрах 20
3.7 Перспективы развития 20

4 ТЕСТИРОВАНИЕ ПРОДУКТА 21
4.1 Описание методологии тестирования 21
4.2 Функциональное тестирование 22
4.3 Тестирование в реальных условиях 23
4.4 Анализ полученных результатов тестирования 25

ЗАКЛЮЧЕНИЕ 25

А ПРИЛОЖЕНИЕ 28
А.1 Результаты тестирования приложения 28

СПИСОК СОКРАЩЕНИЙ И ТЕРМИНОВ

ATT&CK — Adversarial Tactics, Techniques, and Common Knowledge — база знаний MITRE по тактикам и техникам атак злоумышленников.

C# — объектно-ориентированный язык программирования платформы .NET, используемый для разработки приложения.

CSV — Comma-Separated Values — текстовый формат хранения табличных данных, применяемый для экспорта истории операций.

D3FEND — база знаний MITRE по защитным техникам и мерам противодействия компьютерным угрозам.

GUI — Graphical User Interface — графический пользовательский интерфейс приложения.

HTTP — HyperText Transfer Protocol — протокол передачи гипертекста; в современных пользовательских сценариях сам по себе не должен рассматриваться как основная угроза без учёта HTTPS и локальных атак.

HTTPS — HyperText Transfer Protocol Secure — защищённая версия HTTP с использованием криптографической защиты соединения.

MITRE — некоммерческая организация, разрабатывающая и поддерживающая прикладные модели и базы знаний по информационной безопасности.

OpenSSL — открытый криптографический программный пакет и набор утилит, используемый в проекте как внешний криптографический движок.

PDF — Portable Document Format — формат электронных документов, используемый для итогового представления пояснительной записки.

RSA — криптосистема с открытым ключом, применяемая для генерации

асимметрических ключей в приложении.

WPF — Windows Presentation Foundation — технология построения графического интерфейса desktop-приложения Windows.

ВВЕДЕНИЕ

Актуальность разработки прикладных средств защиты данных для непрофессиональных пользователей остаётся высокой. Злоумышленники постоянно ищут новые способы компрометации систем и данных пользователей. Одним из таких способов является внедрение вредоносного ПО или перехват пользовательски данных.

Здоумышленник может перехватить данные пользователя, или подменить их при загрузке на свои, похожие, но содержащие внутри вредоносное программное обеспечение. Способом защиты от таких атак является шифрование данных и проверка подленности скачанных файлов.

В рамках данного проекта разрабатывается приложение, работающее поверх библиотеки `OpenSSL`. Приложение умеет шифровать/расшифровывать файлы, генерировать RSA-ключи, вычислять разные хеш-функции, а также предоставляет удобный и понятный интерфейс. Отдельное внимание уделено снижению рисков, которые возникают на этапе использования криптографических средств: при запуске внешнего процесса, обработке путей к файлам, выборе слабых алгоритмов, подмене исполняемого файла криптографической библиотеки и при возможном дальнейшем подключении расширений.

Особенностью проекта является ориентация не только на реализацию базового функционала, но и на проектирование безопасной архитектуры и развития. Приложение имеет поддержку плагинов и расширений: смена тем оформления, добавление новых алгоритмов, внедрение дополнительных проверок паролей, расширение журналирования, включение новых модулей анализа файлов.

Целью моего проекта является разработка безопасного и расширяемого криптографического приложения обеспечивающего базовые операции защиты данных для домашних, учебных и офисных сценариев использова-

ния.

Задачи проекта:

1) реализовать основные криптографические функции;
2) реализовать журналирование истории операций и проверку паролей;
3) разработать пользовательский интерфейс приложения;
4) создать механизм поддержки расширений и плагинов;

1 ИССЛЕДОВАНИЕ ПРОБЛЕМЫ

1.1 Предметная область

Предметной областью проекта являются пользовательские программные средства защиты информации. К таким средствам относятся программы, позволяющие:

1) шифровать и расшифровывать файлы;
2) вычислять криптографические хеш-функции;
3) проверять целостность скачанных файлов по хешу;
4) генерировать ключевой материал;
5) сохранять историю выполненных операций;
6) предоставлять понятный и простой интерфейс.

В отличие от корпоративных систем защиты информации, где безопасность обеспечивается комплексом централизованных средств, домашний пользователь принимает решения самостоятельно. Он скачивает файлы с разных непроверенных сайтов, отправляет данные по ненадежным источникам и не всегда способен оценить риски. Поэтому важно иметь под рукой приложение, способное повысить безопасность пользователя.

1.2 Проблема и практическая значимость

Основная проблема заключается в том, что у многих пользователей отсутствует удобный инструмент, который одновременно:

- скрывает лишнюю сложность консольной работы;
- делает криптографические операции наглядными;
- может шифровать данные для безопасной отправки;
- предупреждает о слабых паролях и небезопасных настройках;
- позволяет проверять целостность исполняемых файлов и пользовательских данных;
- проектируется с учётом реальных современных угроз, а не устаревших сценариев.

Важно подчеркнуть, что в актуальных условиях простая угроза как перехват HTTP-трафика для подобного проекта не является современной. Большинство уже используют HTTPS, однако злоумышленник может например подменить dns и перенаправить пользователя на сайт, использующий http. Также не стоит забывать такие угрозы как подмена бинарных файлов, запуск неподконтрольных процессов, ошибки в аргументах командной строки, небезопасные локальные расширения, утечки через журналы и использование слабых параметров безопасности.

1.3 Сравнение с существующими аналогами

Таблица 1: Сравнение разрабатываемого приложения с аналогами

Аналог	**Преимущества**	**Недостатки**	**Отличие моего проекта**
CyberChef	Удобный интерфейс, много операций	Не ориентирован на локальное файловое шифрование	Ориентирован на работу с файлами и расширяемость
OpenSSL CLI	Гибкость, много алгоритмов	Сложен для обычного пользователя	Проект даёт GUI-обёртку и упрощает использование OpenSSL
Kleopatra	Удобная работа с ключами и сертификатами	Узкая специализация. Создан для управления PGP сертификатами	Проект сочетает шифрование файлов, хеширование и работу с ключами
CrypTool 2	Богатый набор криптографических функций	Сложен для повседневного применения	Проект проще и удобнее для обычного пользователя
GitHub-проекты	Открытый код, готовые идеи	Часто заброшены и ограничены по функциям	Проект изначально ориентирован на развитие, тестирование и поддержку
VeraCrypt	Надёжная защита носителей	Не решает задачи хеширования и GUI-обёртки над OpenSSL	Проект предназначен для более широкого набора криптографических операций

По результатам сравнения можно сделать вывод, что существующие аналоги либо сложны для неподготовленного пользователя, либо решают только часть нужных задач. Разрабатываемый продукт объединяет удоб-

ный интерфейс, базовые криптографические функции и возможность дальнейшего расширения.

1.4 Аудитория проекта

Мое приложение актуально для любого пользователя, кто хотя бы один раз скачивал файлы или отправлял какие-либо личные данные. Это большая часть всех пользователей интернета по всему ммиру, которые не имеют инструмента для защиты свои данных. Особую значимость приложение представляет людям, не связанным с ИБ и большими технологическими компаниями, где специалисты сами следят за вашей безопасностью.

Еще одна важная группа пользователей - представители малого бизнеса, не имеющие команды специалистов. Приложение доступно каждому и предоставляет надежный способ защитить важные данные.

2 РЕШЕНИЕ ПРОБЛЕМЫ

2.1 Описание разрабатываемого приложения

2.1.1 Общее назначение системы

Приложение представляет собой desktop-программу для операционной системы Windows, написанную на языке C#. В качестве внешнего криптографического инструмента используется `openssl.exe`. Программа предназначена для выполнения операций локальной защиты данных и поддержки безопасных сценариев повседневного использования. Она должна повышать безопасность пользователя.

2.1.2 Основные функции

Приложение включает в себя следующие функции:

1) шифрование файлов с выбором алгоритма;

2) расшифрование файлов;

3) оценка сложности пароля;

4) вычисление хеша файла;

5) сравнение двух файлов по хешу;

6) генерация RSA-ключей заданной длины;

7) просмотр истории операций;

8) экспорт истории;

9) переключение светлой и тёмной темы;

10) отображение команд `OpenSSL`, используемых приложением;

11) настройки безопасности и приватности;

12) проверка наличия бинарника openssl.exe при запуске;

13) проверка целостности бинарного файла openssl.exe по хешу;

14) архитектурная поддержка плагинов и расширений.

2.2 Функциональные возможности проекта

2.2.1 Шифрование и расшифрование файлов

Пользователь может выбрать файл, указать пароль, определить алгоритм шифрования и при необходимости выключить режим использования соли. На основе выбранных параметров формируется команда которая передается в openssl. Результат операции выводится в журнал и на экран.

Поддержка нескольких алгоритмов расширяет возможности пользователя. В интерфейсе также возможно предупреждение о слабых алгоритмах и небезопасных сочетаниях параметров. Кроме международно распространённых алгоритмов, архитектура приложения предусматривает возможность подключения отечественных алгоритмов, таких как «Кузнечик» через механизм расширений, что важно с точки зрения ориентации на российские требования в области информационной безопасности и использования доверенных криптографических средств.

2.2.2 Проверка сложности пароля

Важной функцией является оценка сложности пароля до шифрования. Такой подход снижает вероятность того, что пользователь зашифрует файл слабым паролем, который будет легко подобрать. Возможность принудительно разрешить слабый пароль оставлена только как осознанное действие пользователя и сопровождается дополнительным подтверждением.

2.2.3 Вычисление и сравнение хешей

Приложение умеет вычислять хеш файла и сравнивать два файла по хешу. Это полезно в двух типовых сценариях:

1) проверка целостности скачанного файла по опубликованному значению хеша;

2) проверка совпадения или различия двух локальных копий файла.

Практическая ценность этой функции состоит в защите от подмены или повреждения файла. Если пользователь скачал программу, образ или архив и знает ожидаемое значение хеша, он может убедиться, что файл не был изменён.

2.2.4 RSA

Приложение позволяет генерировать RSA-ключи разной длины и разные сертификаты.

2.2.5 История операций

История операций отображается в виде таблицы и может экспортироваться в CSV. Это позволяет проанализировать, какие действия выполнялись, какие алгоритмы использовались и какие результаты были получены. В дальнейшем подсистема истории может быть расширена функциями фильтрации, очистки по сроку хранения и отдельного режима приватности.

2.2.6 Настройки

В интерфейсе предусмотрен режим настроек, в котором пользователь может задать параметры по умолчанию, путь к `OpenSSL`, правила журналирования и значения, связанные с безопасностью и т.д.

2.2.7 Поддерживаемые и планируемые расширения

Одним из ключевых преимуществ проекта является его ориентированность на расширяемость. Планируется поддержка модулей следующих типов:

- плагины тем оформления;
- плагины новых алгоритмов или режимов шифрования;
- плагины генераторов паролей;
- плагины проверки паролей, например по словарям утёкших паролей;

Система проектируется как базовая платформа, способная адаптироваться к новым требованиям без полного переписывания приложения.

2.3 Модель угроз

Данный проект направлен на нейтрализацию не конкретной, а модели аттак, а совокупности разных угроз. Для формального описания угроз и механизмов противодействия им используется структура фреймворка MITRE ATT&CK, что позволяет четко определить, от чего именно защищает приложение.

Общая цель злоумышленника: Перехватить данные пользователя при их отправке для получения информации, доступ к которой он иметь не должен, или подменить файл при скачивании для загрузки вредоносного ПО на компьютер пользователя.

2.3.1 Соответствие угроз техникам MITRE ATT&CK и меры противодействия

T1557 — Adversary-in-the-Middle

Данной технике соответствует угроза перехвата или подмены данных при получении их из внешних источников. В пользовательском сценарии это может быть подмена загружаемого файла, контрольной суммы или ключевого материала, который затем будет обработан приложением. Речь идёт не только о перехвате HTTP-трафика, но и о компрометации источника данных или конечной точки пользователя.

Приложение противодействует этой угрозе за счёт дополнительной проверки целостности. Для этого выполняется вычисление и контроль хеша внешнего компонента `openssl.exe`. Такой механизм позволяет обнаружить подмену исполняемого файла даже при внешне доверенной загрузке.

T1195 — Supply Chain Compromise

Данной технике соответствует угроза подмены внешних зависимостей, используемых приложением. В контексте проекта основной риск связан с возможной модификацией файла `openssl.exe`, который выполняет криптографические операции. Подменённый компонент может изменить результаты обработки данных или повлиять на работу программы.

Приложение противодействует данной угрозе за счёт проверки наличия внешнего компонента и контроля его хеша при запуске. Это позволяет снизить риск использования модифицированного исполняемого файла и повысить доверие к используемому криптографическому модулю.

T1059 — Command and Scripting Interpreter

Данной технике соответствует угроза внедрения нежелательных параметров при запуске внешнего процесса. Поскольку приложение взаимодействует с OpenSSL через создание процесса, небезопасное формирование командной строки может привести к передаче дополнительных аргументов или изменению логики выполнения.

Приложение противодействует этой угрозе за счёт безопасного формирования параметров процесса. Аргументы передаются отдельно через механизм `psi.ArgumentList.Add()`, что снижает вероятность командной инъекции и делает запуск внешнего инструмента более контролируемым.

T1565 — Data Manipulation

Данной технике соответствует угроза несанкционированного изменения файлов или результатов обработки данных. В контексте проекта это может означать подмену исходного файла, изменение зашифрованного

файла или модификацию контрольной суммы.

Приложение противодействует этой угрозе за счёт вычисления и сравнения хешей файлов. Пользователь может проверить контрольную сумму и выявить факт изменения данных, что особенно важно при передаче файлов и проверке их целостности.

T1027 — Obfuscated/Encrypted Files or Information

Данной технике соответствует использование шифрования для скрытия содержимого данных. В рассматриваемой модели угроз это связано с защитой пользовательских файлов, которые могут содержать конфиденциальную информацию.

Приложение противодействует этой угрозе за счёт функций шифрования и расшифрования файлов. Использование криптографических алгоритмов делает содержимое данных недоступным без знания ключа или пароля и снижает риск раскрытия информации.

T1204 — User Execution

Данной технике соответствует ситуация, когда безопасность зависит от действий пользователя. Пользователь может выбрать слабый пароль, использовать недоверенный файл или проигнорировать проверку целостности данных.

Приложение снижает влияние человеческого фактора с помощью защитных механизмов. К ним относятся проверка сложности пароля, возможность сравнения хешей файлов и контроль наличия внешних зависимостей.

T1499 — Endpoint Denial of Service

Данной технике соответствует угроза нарушения работоспособности приложения из-за отсутствия критической зависимости. В данном проекте такой сценарий возможен при запуске программы без файла `openssl.exe`.

Приложение противодействует этой угрозе за счёт проверки наличия внешнего компонента при запуске. При его отсутствии программа сообщает об ошибке и предотвращает выполнение операций в некорректном состоянии.

2.3.2 Вывод по модели угроз

Моё приложение противодействует множеству техник и является отличной защитой против большого количества атак. Оно лишает злоумышленника возможности подменивать файлы или читать то что пользователь передаёт в случае перехвата трафика, а значит обеспечивает конфиденциальность данных пользователя.

3 АРХИТЕКТУРА И ПРИНЦИП РЕАЛИЗАЦИИ

3.1 Техническое задание

3.1.1 Цели и задачи разработки

Целью разработки является создание криптографического приложения для домашних, офисных и учебных пользователей, которое позволит выполнять базовые операции защиты данных через удобный графический интерфейс. Продукт должен снизить порог входа в использование криптографических инструментов и предоставить пользователю возможность выполнять типовые операции без необходимости ручной работы с консолью и сложными параметрами командной строки.

3.1.2 Технические требования

В качестве языка программирования для реализации приложения выбран **C#**. Выбор данного языка обусловлен тем, что он предоставляет развитые средства для создания приложений, обладает строгой типизацией, удобными механизмами обработки исключений, встроенной поддержкой объектно-ориентированного подхода и хорошо подходит для разработки программ под операционную систему Windows, т.к. был разработан Microsoft специально для этого. Кроме того, C# позволяет сравнительно быстро создавать прикладные интерфейсные решения и поддерживать их дальнейшее развитие.

В качестве основного фреймворка пользовательского интерфейса ис-

пользуется **WPF**. Данный фреймворк выбран потому, что он предназначен для разработки современных приложений с графическим интерфейсом, поддерживает описание интерфейса через XAML, удобную работу со стилями, шаблонами, ресурсами и визуальными состояниями. Это особенно важно для проекта, в котором предусматриваются разные режимы работы, переключение панелей, поддержка тем оформления и возможное расширение интерфейса в дальнейшем.

Для выполнения криптографических операций используется внешний инструмент **OpenSSL**. Его выбор обусловлен тем, что OpenSSL является широко известной и практически значимой библиотекой и набором утилит, поддерживающим большое количество криптографических алгоритмов, режимов шифрования, хеш-функций и механизмов генерации ключей. Использование OpenSSL позволяет не реализовывать криптографические примитивы вручную, а опираться на уже существующий и широко применяемый инструментарий. Это повышает практическую ценность проекта и делает приложение ближе к реальным сценариям применения. Дополнительно такой подход упрощает расширение функциональности, в том числе добавление новых алгоритмов.

В качестве средств разработки использовалась среда **Visual Studio**. Для хранения истории операций были выбраны стандартные средства работы с файлами и сериализацией данных, такие как формат JSONL или CSV. Такой выбор оправдан простотой реализации, удобством отладки и возможностью последующего анализа действий пользователя.

Архитектура продукта должна предусматривать модульность. Это означает, что отдельные компоненты приложения, связанные с темами оформления, подключаемыми алгоритмами, дополнительными режимами обработки данных или экспортом результатов, должны разрабатываться с учётом возможности дальнейшего расширения. Такой подход обеспечивает масштабируемость проекта и делает его пригодным не только для текущей версии, но и для последующего развития.

3.1.3 Описание выполняемых функций продукта

Разрабатываемый программный продукт должен обеспечивать выполнение набора базовых функций, связанных с защитой данных и кон-

тролем их целостности. В общем виде приложение должно предоставлять пользователю возможность выбрать требуемый режим работы и выполнить соответствующую операцию через графический интерфейс.

К основным функциям продукта относятся:

- шифрование файлов с использованием выбранного алгоритма;
- расшифрование ранее зашифрованных файлов;
- вычисление хеш-значений для файлов или текстовых данных;
- сравнение хешей для проверки целостности;
- генерация криптографических ключей;
- отображение информации о выполняемой операции;
- сохранение истории действий пользователя;
- поддержка изменения внешнего вида приложения;
- поддержка расширений и плагинов для добавления новых возможностей.

Функции продукта должны быть реализованы таким образом, чтобы пользователь мог выполнять типовые операции без необходимости ручного ввода сложных команд OpenSSL. При этом приложение должно не только запускать нужную операцию, но и обеспечивать базовую проверку корректности условий её выполнения, например наличие внешней зависимости, корректность параметров и безопасную передачу аргументов внешнему процессу.

3.2 Логическая структура приложения

С логической точки зрения приложение выглядит следующим образом:

1) модуль интерфейса пользователя;
2) модуль запуска внешнего процесса `OpenSSL`;

3) модуль оценки сложности пароля;

4) модуль журналирования;

5) модуль проверки целостности бинарной зависимости;

6) модуль настроек;

7) модуль загрузки и управления плагинами.

Модуль интерфейса - основной. Он отвечает за выбор режима работы, ввод путей к файлам, выбор алгоритма и отображение результата. Модуль запуска внешнего процесса выполняет наиболее критичную с точки зрения безопасности роль, поскольку именно в нём осуществляется формирование аргументов и вызов `openssl.exe`. Модуль журналирования сохраняет сведения об операциях без хранения секретов. Модуль проверки целостности контролирует доверенность внешнего бинарного файла.

3.3 Организация пользовательского интерфейса

Интерфейс приложения включает отдельные панели для пяти основных режимов: шифрование/расшифрование, хеширование, генерация ключей, история и управление плагинами. Дополнительно предусмотрена панель настроек, где размещаются параметры безопасности, приватности, путей к внешним компонентам и значений по умолчанию.

3.4 Взаимодействие с внешним процессом

При выполнении криптографической операции приложение создаёт объект `ProcessStartInfo`, запускает `openssl.exe`, перенаправляет стандартный вывод и поток ошибок, а затем ожидает завершения процесса.

3.5 Журналирование и приватность

Приложение ведёт историю операций и поддерживает экспорт данных в CSV. Это полезно для анализа действий, воспроизводимости результатов и учебной демонстрации. Однако журналирование само по себе по-

рождает риски утечки информации. Поэтому в архитектуре зафиксированы следующие принципы:

- пароли не должны сохраняться в журнале;
- при необходимости журнал может хранить только имена файлов без полных путей;
- уровень детализации журналов должен регулироваться настройками;
- журнал следует очищать или ограничивать по сроку хранения.

3.6 Меры защиты, заложенные в проект

3.6.1 Защита от инъекции аргументов

Одним из наиболее важных реализованных решений является отказ от прямого сложения строк при формировании команды запуска `OpenSSL`. Вместо этого используется добавление каждого аргумента отдельно через `psi.ArgumentList.Add()`.

Такой подход уменьшает риск того, что специальные символы, пробелы или специально подготовленные части пользовательского ввода изменят структуру запускаемой команды. Иначе говоря, пользовательский ввод трактуется как значение аргумента, а не как часть синтаксиса командной строки.

3.6.2 Проверка целостности `openssl.exe`

Так как криптографические операции фактически выполняет внешний бинарный файл, вычисление его хеша и сравнение с эталонным значением является важным. Если хеш не совпадает, приложение должно уведомить пользователя и заблокировать выполнение критичных операций до выяснения причин.

3.6.3 Проверка наличия зависимости при запуске

В ходе тестирования был выявлен сценарий отказа: если `openssl.exe` отсутствует, приложение аварийно ломается при попытке выполнить операцию. После выявления проблемы в проект закладывается исправление

— проверка наличия зависимости при старте программы либо до начала первой операции. Если файл отсутствует, пользователь получает понятное сообщение, а не неконтролируемый сбой.

3.6.4 Ограничение утечек через журналирование

В архитектуре проекта фиксируется требование: пароли не должны попадать в журнал. Помимо этого, пользователь должен иметь возможность ограничить журналирование только именами файлов, сократить срок хранения истории и управлять уровнем логирования.

3.6.5 Предупреждения о слабых параметрах

Приложение оценивает сложность пароля и может предупреждать о небезопасных действиях: выборе слабого секрета, использовании устаревших алгоритмов или попытке продолжить работу в рискованном режиме. Подобные меры не заменяют криптографию, но повышают вероятность того, что пользователь выберет безопасные параметры.

3.7 Перспективы развития

Дальнейшее развитие проекта может идти по следующим направлениям:

1) полноценная реализация подсистемы плагинов;
2) проверка подписи и происхождения расширений;
3) расширение набора алгоритмов и режимов шифрования;
4) добавление генераторов паролей и словарных проверок;
5) развитие настроек политики безопасности;
6) добавление отдельного режима верификации скачанных файлов;
7) улучшение обработки ошибок и диагностических сообщений;
8) возможный переход к более строгому архитектурному разделению слоёв приложения.

Особенно важным направлением является безопасная экосистема расширений. Именно она позволит адаптировать приложение под разные сценарии без потери основной архитектурной целостности.

4 ТЕСТИРОВАНИЕ ПРОДУКТА

4.1 Описание методологии тестирования

Тестирование продукта в рамках данного проекта направлено не только на подтверждение работоспособности пользовательских функций, но и на проверку устойчивости приложения к ошибочным действиям пользователя, сбоям внешних зависимостей и ряду прикладных угроз безопасности. Такой подход обоснован тем, что программа работает с пользовательскими файлами, запускает внешний процесс `OpenSSL` и в дальнейшем должна поддерживать механизм плагинов и расширений.

Nестирования строится на сочетании четырёх проверок:

1) проверка базовой функциональности интерфейса и основных режимов работы;

2) проверка корректности реакции на ошибочные и пограничные сценарии;

3) проверка защитных механизмов, связанных с безопасным вызовом внешнего процесса и контролем зависимостей;

4) оценка поведения приложения в условиях, близких к реальной эксплуатации.

Ожидания от тестирования следующие:

- приложение должно корректно выполнять основные криптографические операции при наличии всех необходимых компонентов;

- пользователь должен получать предсказуемый и понятный результат, а не аварийное завершение программы;

- ввод пользователя не должен позволять изменить структуру запускаемой команды;

- приложение должно обнаруживать признаки подмены внешнего криптографического бинарного файла;
- приложение должно выявлять отсутствие критически важной зависимости `openssl.exe`;
- журналирование не должно приводить к утечке паролей и иной избыточной чувствительной информации;

Таким образом, тестирование рассматривается как средство подтверждения не только функциональной готовности, но и минимально необходимой безопасности продукта.

4.2 Функциональное тестирование

Функциональные проверки проводились в формате сопоставления ожидаемого результата и фактически полученного поведения приложения.

Тест 1. Шифрование существующего файла.

Ожидаемое: приложение принимает путь к существующему файлу, пароль и выбранный алгоритм, после чего формирует корректный вызов `OpenSSL` и создаёт зашифрованный файл.

Получено на практике: базовый сценарий шифрования отрабатывает корректно, операция запускается через графический интерфейс, а результат сохраняется в целевой файл.

Тест 2. Расшифрование ранее обработанного файла.

Ожидаемое: при вводе корректного пароля приложение должно восстановить исходные данные без повреждения структуры файла.

Получено на практике: сценарий расшифрования выполняется корректно для ранее зашифрованных файлов при совпадении параметров и правильном пароле.

Тест 3. Вычисление хеша файла.

Ожидаемое: приложение должно вычислить хеш указанного файла и отобразить его пользователю без изменения самого файла.

Получено на практике: вычисление хеша выполняется штатно, результат отображается в интерфейсе и может использоваться для последующего сравнения.

Тест 4. Сравнение файлов по хешу.

Ожидаемое: для одинаковых файлов приложение должно показать совпадение, а для различных — различие.

Получено на практике: сравнение одинаковых и разных файлов работает ожидаемо, что подтверждает корректность модуля хеширования и проверки целостности.

Тест 5. Генерация RSA-ключей.

Ожидаемое: по запросу пользователя приложение должно создать ключи заданной длины и сохранить их в корректном формате.

Получено на практике: генерация ключей в базовом сценарии выполняется корректно; функция пригодна для учебной демонстрации и дальнейшего развития.

Тест 6. Экспорт истории операций в CSV.

Ожидаемое: журнал операций должен выгружаться в отдельный CSV-файл для последующего просмотра.

Получено на практике: экспорт истории реализуем и позволяет получить структурированный текстовый файл с результатами операций.

Тест 7. Переключение темы оформления.

Ожидаемое: приложение должно менять тему интерфейса без нарушения логики работы.

Получено на практике: переключение темы поддерживается на уровне интерфейса и демонстрирует готовность архитектуры к дальнейшему развитию системы плагинов оформления.

4.3 Тестирование в реальных условиях

Это тестирования, включающие ошибочные действия, нестандартные условия запуска и проверку защитных механизмов.

Сценарий 1. Попытка повлиять на вызов внешнего процесса через пользовательский ввод.

Ожидаемое: пользовательский ввод не должен интерпретироваться как часть синтаксиса командной строки и не должен позволять внедрить дополнительные параметры или изменить смысл команды.

Получено на практике: на раннем этапе риск действительно существовал при подходе с прямым сложением строк. После перехода к пе-

редаче аргументов через `psi.ArgumentList.Add()` данный сценарий был устранён: ввод пользователя передаётся как отдельные аргументы, а не как единая строка команды.

Сценарий 2. Попытка запуска приложения с подменённым `openssl.exe`.

Ожидаемое: приложение должно иметь возможность выявить несоответствие доверенного бинарного файла по хешу и предупредить пользователя о риске.

Получено на практике: в проект заложен механизм вычисления и сравнения хеша `openssl.exe`. Это позволяет обнаруживать подмену внешнего криптографического компонента и использовать данный тест как практическую проверку целостности зависимости.

Сценарий 3. Запуск без `openssl.exe`.

Ожидаемое: отсутствие критически важной зависимости не должно приводить к неуправляемому сбою; приложение должно заранее уведомить пользователя о проблеме.

Получено на практике: в ходе испытаний было установлено, что на ранней стадии приложение ломалось при отсутствии `openssl.exe`. После анализа причины было принято решение добавить проверку наличия внешней зависимости при запуске программы.

Сценарий 4. Проверка журналирования.

Ожидаемое: история операций не должна содержать пароли или иную критически важную информацию.

Получено на практике: Пароли не сохраняются, вся конфиденциальная информация остаётся вне истории.

Сценарий 5. Использование слабых параметров безопасности.

Ожидаемое: приложение должно предупреждать пользователя при выборе слабого пароля или небезопасного алгоритма.

Получено на практике: механизм оценки сложности пароля работает корректно, при слабом пароле пользователь не может зашифровать файл пока не подтвердит действие.

4.4 Анализ полученных результатов тестирования

По результатам тестирования можно сделать несколько выводов.

Базовая функциональность продукта подтверждена: приложение способно выполнять ключевые пользовательские операции через графический интерфейс, что соответствует поставленным целям проекта.

Тесты выявили действительно важные прикладные проблемы безопасности. Наиболее значимой оказалась потенциальная опасность небезопасного формирования командной строки. После замены прямой конкатенации строк на использование `psi.ArgumentList.Add()` была реализована более безопасная схема взаимодействия с внешним процессом.

Испытания показали необходимость обязательного контроля внешних зависимостей. Сценарий запуска без `openssl.exe` сначала приводил к поломке программы, однако эта проблема была обнаружена на раннем этапе и исправлена путём проверки наличия зависимости при старте приложения.

Тесты подтвердили практическую ценность контроля целостности внешнего бинарного файла. Проверка хеша `openssl.exe` усиливает доверие к используемому криптографическому компоненту и позволяет противодействовать угрозе подмены.

Тестирование подтвердило жизнеспособность продукта и одновременно позволило определить направления дальнейшего улучшения: усиление диагностики, расширение автоматизированных проверок и безопасная реализация расширяемой архитектуры.

ЗАКЛЮЧЕНИЕ

В результате выполненной работы разработано приложение, позволяющее выполнять базоыве криптографические операции пользователям любого уровня. Оно предотвращает множество угроз и защищает конфиденциальность данных пользователя. Основная цель работы была достигнута. Было проведено множество тестов доказывающих работоспособность продукта и его надежность, а также соответвсие требованиям по функциональности.

Особое внимание в проекте уделено безопасности прикладной интеграции. Рассмотрены и учтены разные современные угрозы: инъекция аргументов командной строки, подмена внешнего бинарного файла, утечки через журналирование, использование слабых параметров безопасности, ошибки, связанные с отсутствием обязательной зависимости, и риски будущей подсистемы расширений. Для описания угроз и мер защиты использованы профессиональные модели MITRE ATT&CK и D3FEND.

Практический результат работы состоит в создании расширяемого приложения, ориентированного на домашнего или учебного пользователя. Реализованные и запланированные решения показывают, что даже при использовании внешней консольной криптографической утилиты можно построить более безопасную, понятную и расширяемую среду работы с криптографией.

СПИСОК ЛИТЕРАТУРЫ

1. OpenSSL Documentation. –
URL: https://docs.openssl.org/3.6/man1/openssl/

2. OpenSSL dgst. –
URL: https://docs.openssl.org/3.2/man1/openssl-dgst/

3. OpenSSL genpkey. –
URL: https://docs.openssl.org/3.4/man1/openssl-genpkey/

4. Microsoft Learn. Windows Presentation Foundation (WPF). –
URL: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/

5. Microsoft Learn. C#. –
URL: https://learn.microsoft.com/en-us/dotnet/csharp/

6. MITRE ATT&CK. Adversary-in-the-Middle (T1557). –
URL: https://attack.mitre.org/techniques/T1557/

7. MITRE ATT&CK. Supply Chain Compromise (T1195). –
URL: https://attack.mitre.org/techniques/T1195/

8. MITRE ATT&CK. Command and Scripting Interpreter (T1059). –
URL: https://attack.mitre.org/techniques/T1059/

9. MITRE D3FEND. File Integrity Monitoring (D3-FIM). –
URL: https://d3fend.mitre.org/technique/d3f:FileIntegrityMonitoring/

10. MITRE D3FEND. System File Analysis (D3-SFA). –
URL: https://d3fend.mitre.org/technique/d3f:SystemFileAnalysis/

11. Указ Президента Российской Федерации от 01.05.2022 № 250 «О дополнительных мерах по обеспечению информационной безопасности Российской Федерации». –
URL: https://publication.pravo.gov.ru/Document/View/0001202205010023

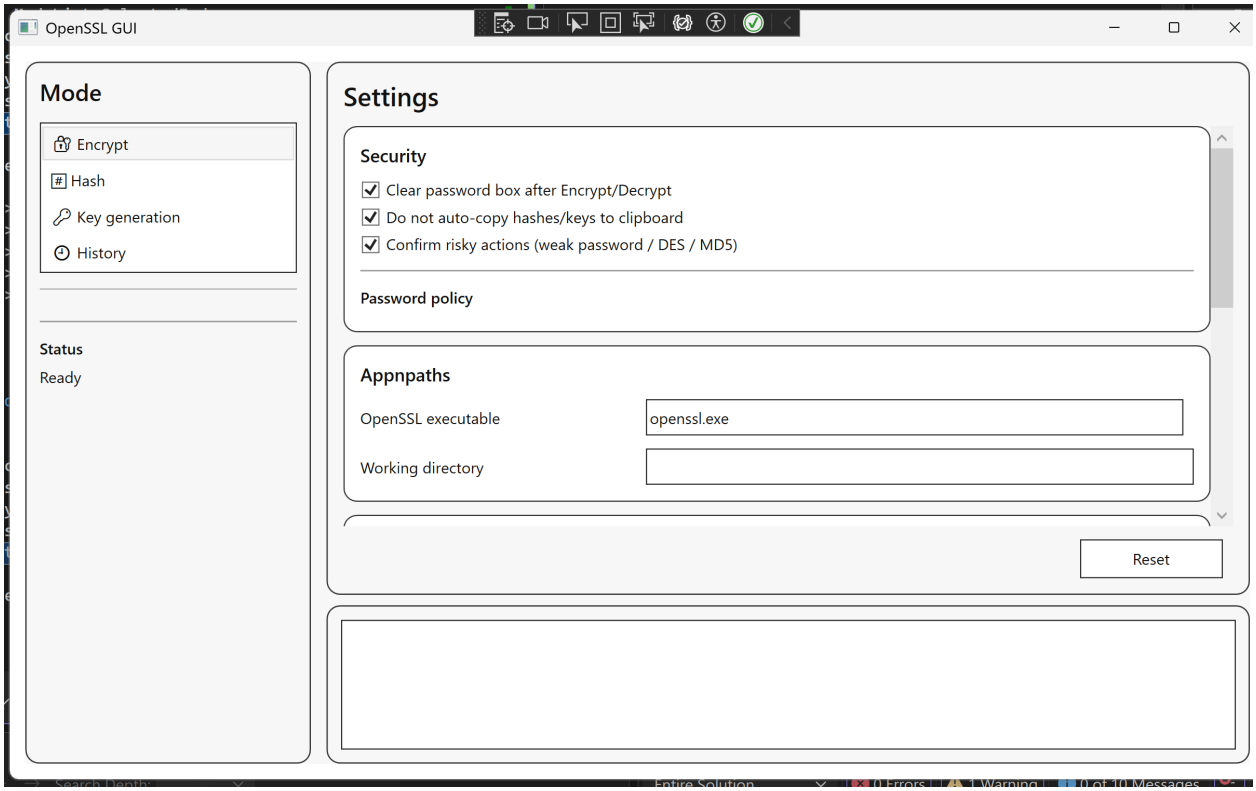
12. Введение в OpenSSL: основы криптографии. –
URL: https://habr.com/ru/companies/otus/articles/666700/

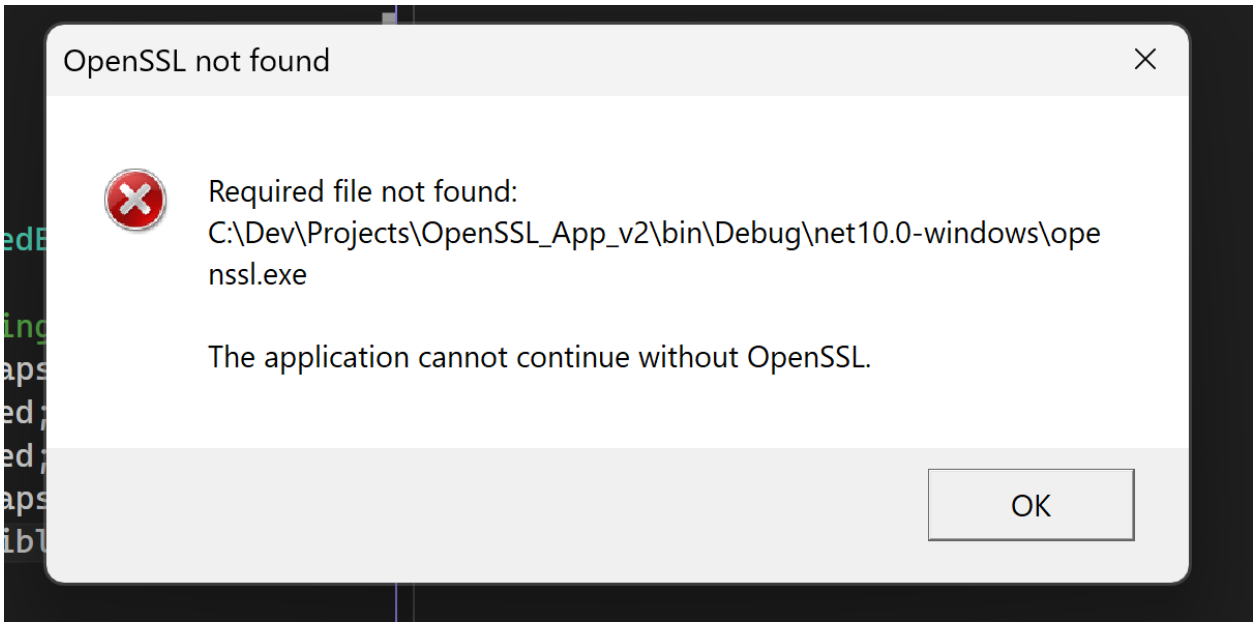
13. 20+ open source утилит для шифрования файлов на все случаи жизни. –
URL: https://habr.com/ru/companies/bastion/articles/711064/

14. Шнайер Б. Прикладная криптография: протоколы, алгоритмы и исходные тексты на языке C / Bruce Schneier. – New York: John Wiley & Sons, 1996.

А ПРИЛОЖЕНИЕ

А.1 Результаты тестирования приложения





Operation history

Time	Operation	Input	Output	Alg.	Status
2026-01-09 23:35:56	Encrypt	C:\Users\Dave\	C:\Users\Dave\	aes-256-cbc	OK
2026-01-10 00:05:51	Hash	C:\Users\Dave\		sha512	OK
2026-01-10 00:31:32	HashCompare	C:\Users\Dave\	C:\Users\Dave\	sha256	OK
2026-01-10 00:31:38	HashCompare	C:\Users\Dave\	C:\Users\Dave\	sha256	OK
2026-01-10 00:34:10	KeyGen		rsa_key.pem	RSA-4096	OK
2026-01-10 00:34:29	Hash	C:\Users\Dave\		sha512	OK
2026-01-10 00:35:01	Hash	C:\Users\Dave\		sha512	OK
2026-02-27 12:32:21	Hash	C:\Users\alexz\		sha256	OK
2026-03-02 00:05:20	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	OK
2026-03-02 00:05:35	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	OK
2026-03-02 00:35:41	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	ERROR
2026-03-02 00:36:16	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	ERROR
2026-03-02 00:43:39	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	OK
2026-03-12 23:09:31	Encrypt	C:\Users\alexz\	C:\Users\alexz\	aes-256-cbc	OK
2026-03-14 00:04:38	KeyGen		rsa_key.pem	RSA-2048	OK

Encrypt / Decrypt

File

C:\Dev\null

Browse

Password

••••••••

Password strength

Weak

☑ Allow weak password (not recommended)

Encryption algorithm

aes-256-cbc

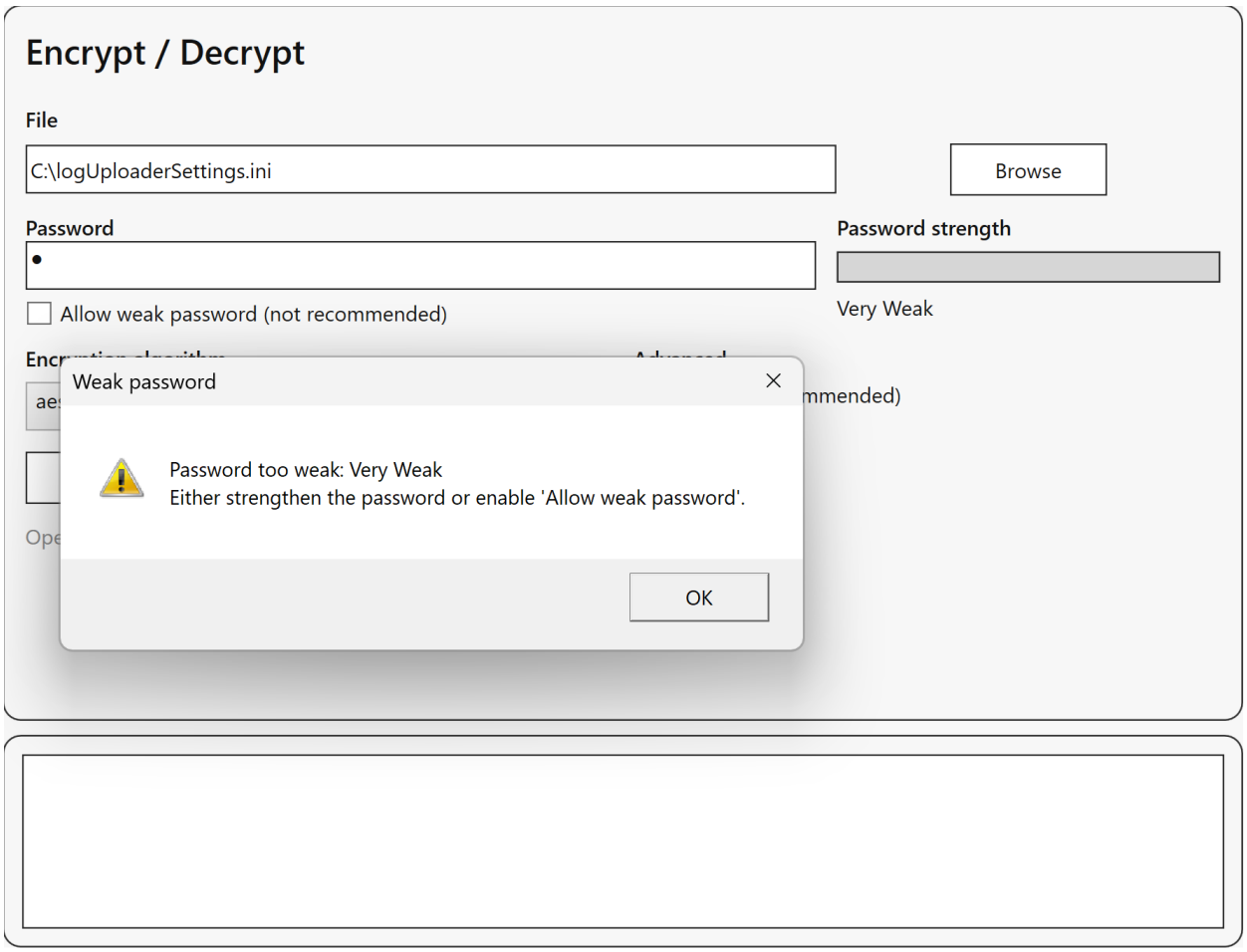
Advanced

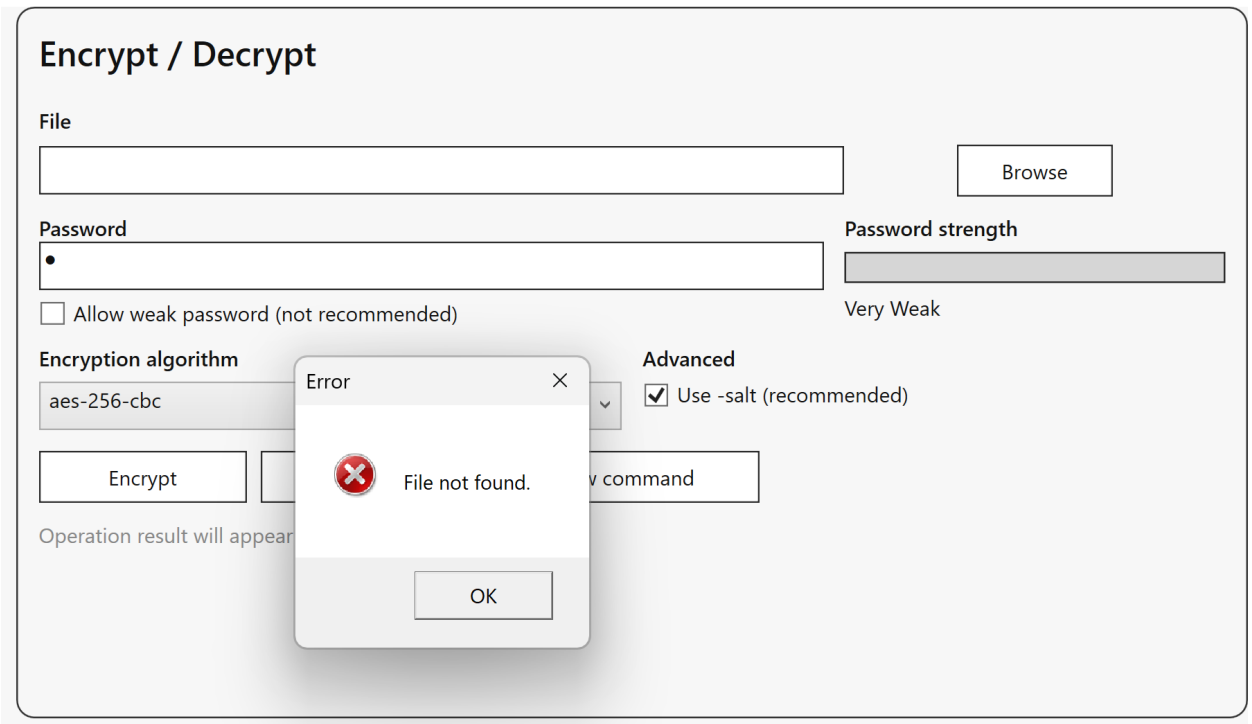
☑ Use -salt (recommended)

Encrypt | Decrypt | Show command

Operation result will appear in the log below.

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
[Encrypt] aes-256-cbc -> C:\Dev\null.enc
```


Encrypt / Decrypt
File
C:\logUploaderSettings.ini
Browse
Password
Password strength
Allow weak password (not recommended)
Very Weak
Weak password
Password too weak: Very Weak
Either strengthen the password or enable 'Allow weak password'.
OK


Encrypt / Decrypt
File
Browse
Password
Password strength
Allow weak password (not recommended)
Very Weak
Encryption algorithm
aes-256-cbc
Advanced
Use -salt (recommended)
Encrypt
Operation result will appear
Error
File not found.
OK